

Controlador difuso para motor CD — Guía del proyecto

Resumen

Diseñamos un controlador de **lógica difusa** que mantiene la velocidad de un motor CD en el valor que le pidamos, aunque le metan una carga. Lo probamos en dos fases:

- **Fase A — Simulación (Python):** el controlador y un modelo matemático del motor corren en la computadora. Sirve para diseñar, sintonizar y comparar Mamdani, Sugeno y un PI clásico.
- **Fase B — ESP32:** el mismo controlador, escrito en C, corre dentro del ESP32. Como no tenemos sensor de velocidad, lo probamos en modo **HIL**: la PC simula el motor, el ESP32 decide el voltaje y ese voltaje también se manda al motor físico para verlo reaccionar.

Resultado principal: el controlador difuso llega a 1500 rpm en 0.19 s **sin sobrepaso**, y el ESP32 da **exactamente** las mismas curvas que Python. Eso demuestra que el código embebido está bien.

Todo el código está en GitHub: [shaz1206/Control_Neurodifuso](https://github.com/shaz1206/Control_Neurodifuso), carpeta [motor_cd_difuso/](#).

Simulación vs HIL

La diferencia es **dónde corre el controlador**. En ambos modos el motor que se "mide" es un modelo matemático en la PC, porque no tenemos sensor.

HIL significa *Hardware-in-the-Loop* ("hardware dentro del lazo"): una parte real (el ESP32) se conecta a una parte simulada (el motor) y trabajan juntas como si todo fuera real.

	Simulación	HIL
¿Dónde corre el controlador difuso?	En la PC (Python)	En el ESP32 (C)
¿Dónde está el motor que se mide?	Modelo en la PC	Modelo en la PC

¿Se mueve el motor físico?	No	Sí, recibe el mismo voltaje
¿Necesita el ESP32?	No	Sí, por USB
¿Para qué sirve?	Diseñar y sintonizar rápido	Probar que el código del ESP32 hace lo mismo que el diseño
Archivos	<code>fase_a/simular.py</code>	<code>fase_b/hil_planta.py</code> o <code>fase_b/interfaz.py</code>

En HIL, cada 10 ms pasa esto:

sequenceDiagram

```

participant PC as PC (modelo del motor)
participant ESP as ESP32 (control difuso)
participant M as Motor físico
PC->>ESP: y1234.5 (velocidad simulada)
ESP->>ESP: calcula el voltaje con el FIS
ESP->>PC: u5.42 (voltaje)
ESP->>M: PWM al puente H
PC->>PC: avanza el modelo 10 ms con ese voltaje

```

La PC manda la velocidad, el ESP32 contesta con el voltaje y la PC calcula cómo reacciona el motor. Si el ESP32 no contesta (por ejemplo, porque se reinició), la prueba se corta con el error "salió del modo HIL".

Lo que HIL no demuestra: que controlamos la velocidad real del motor. El motor físico sigue al modelo en lazo abierto. Si lo frenan con la mano, el controlador no se entera. Para eso haría falta un sensor.

Cómo funciona el controlador difuso

Cada 10 ms el controlador mira **dos cosas** y decide **un cambio de voltaje**:

- **e** = referencia – velocidad (qué tan lejos estamos).
- **de** = e actual – e anterior (si nos estamos acercando o alejando).
- **du** = cuánto subir o bajar el voltaje. El voltaje real es $u = u_{\text{anterior}} + du$, limitado a ± 12 V.

Como se acumula du , el controlador se comporta como un **PI**: elimina el error final. Limitar u en cada paso evita el *windup*.

flowchart LR

```

A[e y de] --> B[Normalizar<br/>Ke, Kde]
B --> C[Fuzzificar<br/>5 conjuntos]
C --> D[25 reglas<br/>AND = min]
D --> E[Defuzzificar<br/>Mamdani o Sugeno]
E --> F[du x Kdu]
F --> G[u = u + du<br/>límite 12 V]
  
```

1. Normalizar. e y de se multiplican por $K_e = 1/455$ y $K_{de} = 1/150$ para quedar entre -1 y 1 . Por ejemplo, 455 rpm de error ya cuentan como "grande".

2. Fuzzificar. Cada valor se traduce a 5 etiquetas: NG (negativo grande), NP (negativo pequeño), Z (cero), PP (positivo pequeño) y PG (positivo grande). Son triángulos que se traslapan al 50 %, así que un valor pertenece a lo más a dos etiquetas y sus grados suman 1. Ejemplo: $e = 0.3$ es 40 % Z y 60 % PP.

3. Reglas. Una tabla de 5×5 del tipo "SI e es PP Y de es NP ENTONCES du es Z" (estamos abajo pero ya subiendo rápido, no empujes más). La fuerza de cada regla es el mínimo de los dos grados.

$e \backslash de$	NG	NP	Z	PP	PG
NG	NG	NG	NG	NP	Z
NP	NG	NG	NP	Z	PP
Z	NG	NP	Z	PP	PG
PP	NP	Z	PP	PG	PG
PG	Z	PP	PG	PG	PG

La diagonal de Z es la línea de equilibrio: si el error y su tendencia se compensan, no se toca el voltaje.

4. Defuzzificar (aquí está la diferencia):

	Mamdani	Sugeno (orden 0)
Salida de cada regla	Un triángulo recortado a la fuerza de la regla	Un número fijo (-1, -0.5, 0, 0.5, 1)
Cómo se combinan	Se unen los triángulos y se saca el centroide	Promedio ponderado: $\Sigma(w \cdot z) / \Sigma w$
Costo en el ESP32	Recorre 201 puntos	Una suma, muy rápido
Salida máxima	≈ 0.83 (el centroide nunca llega a 1)	1
En nuestras pruebas	Filtra mejor el ruido	Rechaza un poco mejor la carga

5. Escalar. $du \text{ normalizado} \times Kdu = 2.5 \text{ V por paso.}$

Ejemplo completo para practicar en el pizarrón ($e = 0.3$, $de = -0.2$): Sugeno da $(0.4 \cdot (-0.5) + 0.6 \cdot 0.5) / 1.8 = \mathbf{0.0556}$, o sea $du \approx 0.14 \text{ V}$. Mamdani da 0.061. El paso a paso está en

[GUIA_DEFENSA.md](#), sección 3.

Código de la Fase A (carpeta **fase_a/**)

Cinco archivos que se importan entre sí; por eso tienen que estar juntos. El que se ejecuta es [simular.py](#).

flowchart TD

```

S[simular.py<br/>corre la prueba] --> C[controlador.py<br/>reglas y ganancias]
S --> M[motor_dc.py<br/>modelo del motor]
C --> F[fuzzy.py<br/>motor de inferencia]
SU[superficie.py<br/>gráficas del FIS] --> C
T[test_controlador.py<br/>10 pruebas] --> S

```

fuzzy.py — el motor de inferencia difusa

Escrito a mano, sin librerías, para poder explicar cada línea.

Parte	Qué hace
<code>trimf(x, a, b, c)</code>	Grado de pertenencia triangular: 0 en a, 1 en b, 0 en c. Si $a = b$ o $b = c$ es un "hombro" que se queda en 1 hacia afuera.
<code>CONJUNTOS_ESTANDAR</code>	Los 5 triángulos NG...PG entre -1 y 1.
<code>SINGLETONS</code>	Los números fijos que usa Sugeno: -1, -0.5, 0, 0.5, 1.
<code>VariableDifusa.fuzzificar(x)</code>	Convierte un número en <code>{etiqueta: grado}</code> .
<code>SistemaDifuso.activaciones(e, de)</code>	Evalúa las 25 reglas con AND = min y devuelve las que se activan.
<code>SistemaDifuso.evaluar(e, de)</code>	Hace todo el proceso y devuelve du. Tiene las dos ramas: Sugeno (promedio ponderado) y Mamdani (recortar, unir con max, centroide).

motor_dc.py — el modelo físico del motor

Dos ecuaciones: la eléctrica ($L \cdot di/dt = V - R \cdot i - K \cdot w$) y la mecánica ($J \cdot dw/dt = K \cdot i - b \cdot w - \text{carga}$).

Parte	Qué hace
<code>__init__</code>	Parámetros de un motor pequeño de 12 V: $R = 2 \, \Omega$, $L = 5 \, \text{mH}$, $K = 0.05$, $J = 5 \times 10^{-5}$.

<code>paso(V, dt, carga)</code>	Avanza el tiempo dt con Runge-Kutta de 4.º orden. Limita V a ± 12 V como el puente H real. Devuelve rpm.
---------------------------------	--

<code>rpm_estacionaria(V)</code>	Velocidad final con V fijo: 2274 rpm con 12 V.
----------------------------------	--

Se usa un paso de 0.1 ms porque la parte eléctrica es muy rápida (2.5 ms) y un paso grande se vuelve inestable.

controlador.py — el controlador

Parte	Qué hace
<code>TABLA_REGLAS / REGLAS</code>	La tabla 5×5 de la sección anterior.
<code>KE, KDE, KDU, TS</code>	Ganancias (1/455, 1/150, 2.5 V) y periodo de 10 ms. Se eligieron con un barrido de valores.
<code>ControladorDifuso.calcular(ref, medida)</code>	Calcula e y de, llama al FIS, acumula u y lo limita a ± 12 V.
<code>ControladorPI</code>	Un PI clásico con anti-windup, solo para comparar.

simular.py — la prueba en lazo cerrado

Parte	Qué hace
<code>referencia(t), carga(t)</code>	El escenario: 1500 rpm, luego 800 rpm a los 1.5 s, y carga de 0.03 N·m a los 2.5 s.

<code>simular(ctrl, motor)</code>	Cada 10 ms el controlador decide u ; entre muestras el motor avanza 100 pasos de 0.1 ms con ese voltaje.
<code>metricas(...)</code>	Sobrepaso, tiempo de subida, tiempo de establecimiento (2 %), error final e IAE.
<code>main()</code>	Compara Mamdani, Sugeno y PI; prueba la robustez (J $\times 2$, R +50 %, K -20 %, ruido) y guarda las gráficas en <code>resultados/</code> .

superficie.py y test_controlador.py

- `superficie.py` dibuja las funciones de pertenencia y las superficies de control, e imprime el ejemplo de inferencia paso a paso.
- `test_controlador.py` tiene 10 pruebas automáticas: triángulos, simetría de las reglas, el ejemplo de Sugeno calculado a mano y que el lazo cerrado siga la referencia. Se corre con `python -m unittest -v`.

Resultados de la simulación

Controlador	Sobrepaso	t. establecimiento	Caída con carga	Sobrepaso con J $\times 2$
Mamdani	0 %	0.19 s	65 rpm	0.6 %
Sugeno	0 %	0.19 s	54 rpm	0.6 %
PI clásico	0 %	0.15 s	101 rpm	8.8 %

El PI es más rápido con el motor nominal, pero el difuso aguanta mejor la carga y casi no cambia si el motor es distinto al modelo.

Código de la Fase B (carpeta `fase_b/`)

Tres piezas: el firmware que va dentro del ESP32 y dos programas de PC para usarlo. `prueba_motor.py` también está ahí, pero solo sirve si algún día conseguimos un sensor.

esp32_fuzzy/esp32_fuzzy.ino — firmware del ESP32 (C++)

Es el mismo controlador de `controlador.py` traducido a C. Se sube con Arduino IDE.

Parte	Qué hace
Configuración (arriba)	Pines (ENA = 27, IN1 = 25, IN2 = 26), <code>HAY_SENSOR = false</code> , <code>RPM_MAX = 2274</code> , <code>ZONA_MUERTA_V = 6.5</code> y las ganancias.
<code>pertenencia(x, j)</code>	Igual que <code>trimf</code> en Python.
<code>evaluarFIS(e, de, mamdani)</code>	Fuzzifica, evalúa las 25 reglas y calcula Sugeno. Si es Mamdani, además calcula el centroide recorriendo 201 puntos.
<code>pasoControl(medida)</code>	Calcula e, de y du, y acumula u (igual que <code>ControladorDifuso.calcular</code>).
<code>aplicarVoltaje(v)</code>	Convierte el voltaje en dirección (IN1/IN2) y PWM de 20 kHz en ENA. Aplica la compensación de zona muerta.
<code>procesarComando(cmd)</code>	Atiende lo que llega por USB (tabla de abajo).
<code>loop()</code>	Lee comandos. En HIL solo espera datos de la PC y apaga el motor si dejan de llegar por 300 ms. En modo normal imprime una línea CSV cada 10 ms.

Comandos que entiende por el puerto serie:

Comando	Qué hace
---------	----------

<code>v6</code>	Lazo abierto: aplica 6 V al motor (negativo = al revés).
<code>p</code>	Paro.
<code>m / s</code>	Usar Mamdani / Sugeno.
<code>h</code>	Entrar o salir del modo HIL.
<code>y1234.5</code> <code>u5.42.</code>	(Solo HIL) La PC le da la velocidad simulada; el ESP32 contesta

Zona muerta: nuestro motor empieza a moverse en unos 7 V por la caída del L298N y la fricción. El firmware traslada el rango 0–12 V del controlador a 6.5–12 V reales. Por ejemplo, 4.2 V pedidos se convierten en unos 8.4 V.

`hil_planta.py` — prueba HIL desde la terminal

Parte	Qué hace
<code>esperar_linea(ser, prefijo)</code>	Lee del USB hasta encontrar una línea que empiece con el texto esperado.
<code>correr_hil(ser, metodo)</code>	Entra a HIL y, durante 4 s, cada 10 ms manda <code>y</code> , recibe <code>u</code> y avanza el modelo del motor.
<code>main()</code>	Al terminar corre la simulación en Python con los mismos datos, compara las dos curvas y guarda PNG y CSV en <code>resultados/</code> .

Uso: `python hil_planta.py --puerto COM7 --metodo mamdani`

`interfaz.py` — todo desde una ventana

Hecha con Tkinter, que viene con Python. Junta la simulación y el HIL con botones.

Parte	Qué hace
Escenario	Guarda las referencias, el momento del cambio y la carga que escribes en la ventana.
correr(...)	El mismo lazo para los dos modos: sin puerto usa el controlador de Python; con puerto usa el ESP32.
esperar_respuesta / diagnosticar	Si el ESP32 no contesta, explica por qué: se reinició, salió de HIL o no llegó nada.
App._panel	Los controles de la izquierda: conexión, modo, método, escenario, acciones y motor manual.
App._trabajo	Corre la prueba en un hilo aparte para que la ventana no se congele.
App._revisarCola	Cada 50 ms toma los datos del hilo y actualiza las gráficas en vivo.
App._registrar	Calcula las métricas de cada corrida y las agrega a la tabla.
App.guardar	Guarda la gráfica (PNG) y los datos (CSV).

Por qué un hilo y una cola: Tkinter solo puede dibujar desde el hilo principal. Si la prueba de 4 s corriera ahí, la ventana se congelaría. Por eso la prueba corre en otro hilo, que deja los datos en una cola, y la ventana los lee cada 50 ms.

Hardware y problemas que encontramos

Usamos un ESP32 sobre una placa de expansión, un puente H L298N, un motor CD con escobillas y una fuente de 12 V. No tenemos sensor de velocidad.

ESP32	L298N	Para qué
GPIO27	ENA (sin jumper)	PWM: cuánto voltaje
GPIO25	IN1	Dirección
GPIO26	IN2	Dirección
GND	GND	Referencia común (obligatorio)
—	12V + GND	Fuente de 12 V
—	OUT1 / OUT2	Motor

Problemas que ya resolvimos, por si vuelven a pasar:

Problema	Causa	Solución
El motor no se movía con nada	Estaba en OUT3/OUT4, que es el canal de ENB/IN3/IN4	Moverlo a OUT1/OUT2
Seguía sin moverse	La placa de expansión tiene otra numeración: los cables estaban en GPIO 25/26/27 pero en otro orden	Programa escáner de pines y cambiar PIN_PWM , PIN_IN1 y PIN_IN2 en el firmware

Con IN1 e IN2 en alto no giraba	El L298N frena el motor cuando ambas entradas están en alto	Pines correctos
Solo gira arriba de ~7 V	Caída del L298N (~2 V) y fricción del motor	<code>ZONA_MUERTA_V = 6.5</code>
"Salió del modo HIL"	El ESP32 se reinicia cuando arranca el motor (ruido o caída de voltaje)	Ver abajo (pendiente)

Para el último problema, en orden:

1. Revisar que el cable rojo empalmado al café vaya del **GND** del L298N al **GND** del ESP32, no a 5V ni a 12V.
2. Soldar un capacitor cerámico de **100 nF** (código 104) entre las dos patas del motor.
3. Conectar el ESP32 directo a la computadora, sin hub.
4. Si sigue, un capacitor electrolítico de 470–1000 µF entre 12V y GND del L298N.

Cómo correr todo

Una vez: clonar el repositorio e instalar las librerías.

```
git clone https://github.com/shaz1206/Control_Neurodifuso.git
cd Control_Neurodifuso/motor_cd_difuso
pip install -r requirements.txt
```

Fase A (sin hardware), dentro de `fase_a`:

1. `python simular.py` — tabla de métricas y gráficas en `resultados/`.
2. `python superficie.py` — superficies de control y el ejemplo de inferencia paso a paso.
3. `python -m unittest -v` — las 10 pruebas.

Fase B (con el ESP32):

1. Abrir `fase_b/esp32_fuzzy/esp32_fuzzy.ino` en Arduino IDE (placa "ESP32 Dev Module") y subirlo.
2. Cerrar el Monitor Serie de Arduino, porque ocupa el puerto.
3. Dentro de `fase_b`, correr `python interfaz.py`.
4. Elegir el puerto (COM7) → *Conectar* → modo *HIL* → *Ejecutar* o *Comparar todos los métodos*.

Error	Qué significa	Qué hacer
No module named 'fuzzy'	Se corrió un archivo suelto, fuera de su carpeta	Trabajar dentro de la carpeta clonada
can't open file ...hil_planta.py	La terminal está en otra carpeta	cd fase_b primero
No se pudo abrir el puerto	El Monitor Serie de Arduino lo está usando	Cerrar el Monitor Serie
"Salió del modo HIL" o "se reinició"	El ESP32 se reinició a mitad de la prueba	Ver la sección de hardware
El HIL sale bien pero el motor no se mueve	El firmware cargado tiene los pines de fábrica	Subir el firmware con ENA = 27, IN1 = 25, IN2 = 26

Para la defensa, cada uno debe poder explicar las secciones de este documento sin leer. Las preguntas probables con sus respuestas están en [GUIA_DEFENSA.md](#).